Tatwir Talent HRMS — Pre-Launch Audit Report

> Generated: 2026-07-18
> Scope: Full codebase deep-scan across all modules (6 parallel audits)
> Status: **NOT production-ready** — 10 critical, 19 high, 28 medium, 19 low issues found

Table of Contents

1. Critical Issues (Fix Immediately)
2. High Severity (Fix Before Launch)
3. Medium Severity (Fix Within First Sprint)
4. Low Severity (Fix Post-Launch)
5. Implementation Plan
6. Fix Priority Matrix

1. Critical Issues

These WILL cause data corruption, financial loss, or security breaches in production.

CRIT-1: Geofencing validated ONLY in browser — employees can fake attendance location

Module: Attendance
Files: `src/app/(app)/attendance/page.tsx:22–30` , `src/lib/services/attendance.ts:36–90`
Impact: Any employee can check in from home while claiming to be at the office

Problem:
The entire geofence calculation (haversine distance, within-radius check) runs in the browser. The `createCheckIn` function writes `within_geofence` , `distance_meters` , `check_in_lat` ,

`check_in_lng` directly to Supabase from the client. There is NO server-side API route between the browser and the DB for attendance. An employee can open DevTools, modify the coordinates, and check in from anywhere.

How to Fix:

1. Create a new API route: src/app/api/attendance/check-in/route.ts
2. Move ALL geofence logic server-side:
   - Accept only { lat, lng } from the client
   - Server fetches employee's allowed locations from employee_locations table
   - Server computes haversine distance
   - Server determines within_geofence boolean
   - Server writes the record with service_role client
3. Remove direct Supabase insert from src/lib/services/attendance.ts
4. Add RLS policy on attendance_records: INSERT only via service_role
5. Optional: Add IP-based rough location validation as secondary check

Effort: 1 day

CRIT-2: employee_id at check-in not verified against authenticated user

Module: Attendance

Files: `src/lib/services/attendance.ts:36-90`

Impact: Employee A can clock in/out Employee B

Problem:

`createCheckIn` receives `employee_id` from the caller and inserts it directly. The function queries `employees` filtered by `eq("id", employeeId)` and `eq("org_id", orgId)` but NOT `eq("user_id", auth.uid())` . Without server-side enforcement, any authenticated user can create attendance records for any colleague in the same org.

How to Fix:

```
1. In the new server-side check-in API route (from CRIT-1 fix):
   - Get auth user from session: const { data: { user } } = await supabase.auth.getUser
   - Query employee WHERE user_id = user.id AND org_id = orgId
   - Use THAT employee's ID for the attendance record (ignore client-supplied employee_
2. Add RLS policy: attendance_records INSERT requires employee_id matches
   current user's employee record via:
   CREATE POLICY attendance_insert ON attendance_records
     FOR INSERT WITH CHECK (
       employee_id = (SELECT id FROM employees WHERE user_id = auth.uid() LIMIT 1)
     );
```

Effort: 0.5 day (part of CRIT-1 fix)

CRIT-3: Race condition in leave balance deduction — double-deduction possible

Module: Leave

Files: `src/lib/services/leaves.ts:564-619` , `src/lib/server/leave-balances.ts:576-618`

Impact: Two concurrent approvals can over-deduct, giving employee free extra leave days

Problem:

Balance deduction follows: (1) READ current balance, (2) calculate new value in JS, (3) WRITE updated value. No `FOR UPDATE` lock, no DB transaction, no atomic operation. Two simultaneous approvals both read `vacation_remaining = 10` , both compute `10 - 3 = 7` , both write `7` — final balance is 7 instead of 4.

How to Fix:

1. Create a PostgreSQL function for atomic deduction:

```
CREATE OR REPLACE FUNCTION deduct_leave_balance(
  p_employee_id uuid,
  p_org_id uuid,
  p_leave_type text,
  p_days numeric
) RETURNS numeric AS $$
DECLARE
  v_current numeric;
  v_new numeric;
BEGIN
  SELECT CASE p_leave_type
    WHEN 'annual' THEN vacation_remaining
    WHEN 'sick' THEN sick_remaining
    WHEN 'personal' THEN personal_remaining
  END INTO v_current
  FROM leave_balances
  WHERE employee_id = p_employee_id AND org_id = p_org_id
  FOR UPDATE;  -- Lock the row

  v_new := v_current - p_days;
  IF v_new < 0 THEN
    RAISE EXCEPTION 'Insufficient leave balance';
  END IF;

  UPDATE leave_balances SET
    vacation_remaining = CASE WHEN p_leave_type = 'annual' THEN v_new ELSE vacation_
    sick_remaining = CASE WHEN p_leave_type = 'sick' THEN v_new ELSE sick_remaining
    personal_remaining = CASE WHEN p_leave_type = 'personal' THEN v_new ELSE persona
    updated_at = now()
  WHERE employee_id = p_employee_id AND org_id = p_org_id;

  RETURN v_new;
END;
$$ LANGUAGE plpgsql;
```

2. Replace all JS-side balance reads + writes with:
   `await supabase.rpc('deduct_leave_balance', { p_employee_id, p_org_id, p_leave_type,`

3. Remove the read-modify-write code in leaves.ts and leave-balances.ts

Effort: 1 day

CRIT-4: No rollback in team member creation — orphaned auth accounts on failure

Module: Sales

Files: `src/app/api/sales/team/route.ts:481–580`

Impact: Failed team creation leaves zombie accounts that can't be re-created

Problem:

The POST handler sequentially: (1) creates auth account, (2) upserts profile, (3) inserts org_member, (4) inserts employee, (5) career event, (6) attendance rows, (7) company assignments. If step 4+ fails, the auth account exists but has no matching records. The next attempt to create the same user triggers CRIT-5 (silent password reset).

How to Fix:

1. Wrap steps 2–7 in a try/catch block
2. On any failure after step 1, call:
   `await adminClient.auth.admin.deleteUser(newUserId)`
3. Return a clear error message to the caller
4. Better approach: Use a Supabase Edge Function or DB transaction:

```
// Pseudo-code for the route handler:
const { data: authUser, error: authError } = await admin.auth.admin.createUser(...)
if (authError) return NextResponse.json({ error: authError.message }, { status: 400

try {
  // All subsequent operations...
  await adminClient.from('profiles').upsert(...)
  await adminClient.from('org_members').upsert(...)
  await adminClient.from('employees').insert(...)
  // etc.
} catch (err) {
  // Rollback: delete the auth user
  await admin.auth.admin.deleteUser(authUser.id)
  return NextResponse.json({ error: 'Failed to create team member' }, { status: 500
}
```

Effort: 0.5 day

CRIT-5: Silent password reset for existing users on team creation

Module: Sales

Files: `src/app/api/sales/team/route.ts:481–487`

Impact: Creating a team member with an existing email silently changes that user's password

Problem:

When a POST request uses an email that already has a Supabase auth account, the code calls `admin.auth.admin.updateUserById(existing.id, { password, ... })` without warning. Any existing user in the same org whose email is reused will have their password changed without notification.

How to Fix:

1. Before creating/updating, check if user already exists in the SAME org:

```
const { data: existingMember } = await adminClient
  .from('org_members')
  .select('id')
  .eq('user_id', existingAuthUser.id)
  .eq('organization_id', orgId)
  .single()

if (existingMember) {
  return NextResponse.json(
    { error: 'This email is already registered as a team member' },
    { status: 409 }
  )
}
```

2. If they exist in a DIFFERENT org, create an org_member link (multi-org support) but NEVER change their password
3. Only set password on brand-new auth accounts

Effort: 0.5 day

CRIT-6: No cancel/void path for SIM sales records

Module: Sales

Files: `src/app/api/sales/merchants/[[...segments]]/route.ts` , `src/app/api/sales/agents/[[...segments]]/route.ts`

Impact: Accidental or fraudulent SIM sales cannot be reversed

Problem:

Once a SIM is marked `sold` and a `sim_sales` row is inserted, there is no API endpoint to reverse or void the sale. The `sim_sales` record, `agent_commissions` row, and `merchant_commissions` row cannot be removed or voided through any route. No correction mechanism exists.

How to Fix:

1. Add a PATCH endpoint to sim_sales for voiding:
   ```
   // In agents/[[...segments]]/route.ts, add a new segment handler:
   case "void-sale":
     // Only allow managers/admins
     if (!auth.isManager && !auth.isAdmin) return forbidden()

     // Update sim_sales status to 'voided'
     await adminClient.from('sim_sales')
       .update({ status: 'voided', voided_at: new Date().toISOString(), voided_by: auth
       .eq('id', saleId)
       .eq('organization_id', auth.orgId)

     // Revert sim_unit status back to 'in_stock' or 'assigned'
     await adminClient.from('sim_units')
       .update({ status: previousStatus })
       .eq('id', simUnitId)

     // Mark commissions as voided (don't delete — audit trail)
     await adminClient.from('agent_commissions')
       .update({ status: 'voided' })
       .eq('sale_id', saleId)
   ```

2. Add 'voided' to the sim_sales status CHECK constraint:
   ```
   ALTER TABLE sim_sales DROP CONSTRAINT sim_sales_status_check;
   ALTER TABLE sim_sales ADD CONSTRAINT sim_sales_status_check
     CHECK (status IN ('pending', 'confirmed', 'rejected', 'voided'));
   ```

3. Add voided_at, voided_by columns to sim_sales
4. Filter out voided sales from all dashboard queries

Effort: 1 day

CRIT-7: No void/cancel state for sales_logs

Module: Sales

Files: `src/app/api/sales/log/route.ts`

Impact: Wrong sales logs cannot be discarded — only edited via request system

Problem:

Status transitions are `submitted → reviewed` only. There is no void, cancelled, or rejected state. A

submitted log that is entirely wrong cannot be voided — only edit-requested. If the edit request system is broken or the log must be entirely discarded, there is no mechanism.

How to Fix:

1. Add 'voided' status to sales_logs:
   ALTER TABLE sales_logs ADD CONSTRAINT IF NOT EXISTS
     sales_logs_status_check CHECK (status IN ('submitted', 'reviewed', 'voided'));

2. Add void endpoint in sales/log/route.ts PATCH handler:
   if (action === 'void') {
     if (!isManager && !isAdmin) return forbidden()
     await adminClient.from('sales_logs')
       .update({ status: 'voided', voided_at: now(), voided_by: userId })
       .eq('id', logId).eq('organization_id', orgId)

     // Reverse the GL auto-posting if it was posted
     if (salesLog.gl_entry_id) {
       await reverseJournalEntry(salesLog.gl_entry_id, 'Sales log voided')
     }
   }

3. Exclude voided logs from all report queries and dashboard aggregations
4. Show voided logs in a separate "Voided" tab with reason

Effort: 1 day

CRIT-8: Auto-posting broken — non-UUID source IDs cause DB failure

Module: Accounting

Files: `src/lib/accounting/auto-posting.ts:159,377,524,766`

Impact: Payroll, depreciation, EOSB, and GOSI journal entries NEVER get created

Problem:

`accounting_journal_entries.source_id` is type `uuid` in the DB. But auto-posting passes string literals like `"auto-payroll-2025-01"`, `"depreciation-2025-01"`, etc. PostgreSQL rejects these with `invalid input syntax for type uuid`. The `logAutoPostFailure` function swallows the error silently.

How to Fix:

```
Option A (Recommended): Change source_id to TEXT type
   ALTER TABLE accounting_journal_entries ALTER COLUMN source_id TYPE text;
   ALTER TABLE accounting_source_postings ALTER COLUMN source_id TYPE text;

Option B: Generate deterministic UUIDs from the string identifiers
   import { v5 as uuidv5 } from 'uuid'
   const NAMESPACE = '6ba7b810-9dad-11d1-80b4-00c04fd430c8' // fixed namespace

   // In auto-posting.ts, replace:
   const sourceId = `auto-payroll-${month}`
   // With:
   const sourceId = uuidv5(`auto-payroll-${month}`, NAMESPACE)

   // This gives a stable UUID per month — idempotent and unique
```

Effort: 0.5 day

CRIT-9: Payroll auto-posting produces unbalanced journal entries

Module: Accounting

Files: `src/lib/accounting/templates/payroll.ts:38-117`

Impact: Payroll GL entries fail for any employee with deductions beyond GOSI/advances

Problem:

The payroll template builds DR/CR lines but `net_salary = base + allowances - gosi - OTHER_DEDUCTIONS`. When `other_deductions > 0`, the credit side is short by that amount. `validateJournalLines()` throws, so the journal entry is never created — silently.

How to Fix:

1. In src/lib/accounting/templates/payroll.ts, add a line for other deductions:

   ```
   // After computing net_salary:
   const otherDeductions = grossSalary - employeeGosi - advanceDeduction - netSalary

   // Add to journal lines:
   if (otherDeductions > 0) {
     lines.push({
       account_id: settings.other_deductions_payable_id, // New setting needed
       description: 'Salary deductions (absence, loans, etc.)',
       credit: otherDeductions,
       debit: 0,
     })
   }
   ```

2. Add `other_deductions_payable_id` to accounting_settings table:

   ```
   ALTER TABLE accounting_settings
     ADD COLUMN IF NOT EXISTS other_deductions_payable_id uuid
     REFERENCES accounting_chart_of_accounts(id);
   ```

3. Verify balance: DR(gross + employer_gosi) = CR(net + emp_gosi + emp_gosi_employer +

Effort: 1 day

2. High Severity

These will cause incorrect data, broken features, or security issues.

HIGH-1: Wrong timezone in DB trigger — Africa/Cairo instead of Asia/Riyadh

Module: Attendance

Files: `db/schema_v14_shift_attendance_timing.sql:54` , `db/schema_v13_shift_late_attendance_alerts.sql:38`

Impact: Late-arrival calculations are wrong by 1 hour during Oct-Apr (Cairo's standard time)

How to Fix:

```
-- Replace in both trigger functions:
-- OLD:
shift_start_at := ((NEW.work_date::timestamp + active_shift.start_time) AT TIME ZONE 'A
-- NEW:
shift_start_at := ((NEW.work_date::timestamp + active_shift.start_time) AT TIME ZONE 'A
```

Effort: 15 minutes

HIGH-2: Attendance unique constraint dropped and never re-added

Module: Attendance

Files: `db/schema_v10_allow_attendance_retests.sql:1-4`

Impact: Duplicate attendance records possible; dead error-handling code

How to Fix:

```
-- Add back the unique constraint (allows one record per employee per day):
ALTER TABLE attendance_records
  ADD CONSTRAINT attendance_records_org_employee_date_uq
  UNIQUE (org_id, employee_id, work_date);

-- The error handler in attendance.ts:85 checking for code "23505" will now work again
```

Effort: 15 minutes (migration) + test existing data for duplicates first

HIGH-3: Device fingerprint in localStorage is trivially spoofable

Module: Attendance

Files: `src/app/(app)/attendance/page.tsx:32-39`

Impact: Device lock provides false sense of security

How to Fix:

1. Short-term: Accept that device fingerprinting is advisory, not security.
   Add a disclaimer in the UI: "Device binding is approximate"

2. Long-term: Move device registration server-side:
   - On first check-in from a device, generate a server-side token (HttpOnly cookie)
   - Store token hash in DB linked to employee
   - On subsequent check-ins, validate the cookie server-side
   - Alert HR if a new device is detected (but still allow check-in)

Effort: 2 days (long-term solution)

HIGH-4: AutoAttendanceRecorder ignores employee_locations table

Module: Attendance

Files: `src/components/attendance/AutoAttendanceRecorder.tsx:49-54`

Impact: Auto-attendance silently disabled for employees using multi-location assignments

How to Fix:

```
// Replace lines 49-54 in AutoAttendanceRecorder.tsx:
// OLD:
const employee = employees.find((emp) => emp.id === currentUser.employeeId);
if (!employee?.location_id) return null;
return locations.find((location) => location.id === employee.location_id) || null;

// NEW:
const employee = employees.find((emp) => emp.id === currentUser.employeeId);
// Check employee_locations first (multi-location), fall back to legacy location_id
const { data: empLocations } = await supabase
  .from('employee_locations')
  .select('location_id')
  .eq('employee_id', employee.id)
  .eq('org_id', orgId);

const locationIds = empLocations?.length
  ? empLocations.map(el => el.location_id)
  : employee?.location_id ? [employee.location_id] : [];

if (!locationIds.length) return null;
return locations.filter(loc => locationIds.includes(loc.id));
```

Effort: 0.5 day

HIGH-5: Commission settlement creates NO GL journal entry

Module: Accounting

Files: `src/app/api/accounting/[...segments]/route.ts:555-617`

Impact: Cash leaves bank account with no corresponding debit — financial statements wrong

How to Fix:

```
// After marking commissions as paid, create the journal entry:
const totalPaid = commissions.reduce((sum, c) => sum + Number(c.amount), 0)

await createJournalEntry(adminClient, {
  orgId: auth.orgId,
  entryDate: new Date().toISOString().split('T')[0],
  description: `Commission settlement — ${commissions.length} records`,
  status: 'posted',
  sourceModule: 'commission_settlement',
  sourceId: settlementId, // Generate a UUID for this batch
  lines: [
    {
      account_id: settings.commission_expense_id, // New setting needed
      description: 'Sales commission expense',
      debit: totalPaid,
      credit: 0,
    },
    {
      account_id: paymentAccountId, // Bank or cash account
      description: 'Commission payment',
      debit: 0,
      credit: totalPaid,
    },
  ],
  createdBy: auth.userId,
})
```

Effort: 1 day

HIGH-6: Advance requests have no limit — unlimited stacking allowed

Module: Accounting

Files: `src/app/api/advance-requests/route.ts:28-92`

Impact: Employee can submit infinite advance requests of any amount

How to Fix:

```
// Add these checks in the POST handler before inserting:

// 1. Check for pending/approved-unpaid advances
const { data: existing } = await supabase
  .from('advance_requests')
  .select('id, requested_amount, status')
  .eq('employee_id', employeeId)
  .eq('org_id', orgId)
  .in('status', ['pending', 'approved'])

const pendingTotal = existing?.reduce((sum, a) => sum + Number(a.requested_amount), 0)

if (pendingTotal > 0) {
  return NextResponse.json(
    { error: 'You have an outstanding advance request. Please wait for it to be resolve
    { status: 400 }
  )
}

// 2. Enforce maximum (e.g., 1 month salary or org-configured limit)
const { data: emp } = await supabase
  .from('employees')
  .select('base_salary')
  .eq('id', employeeId)
  .single()

const maxAdvance = Number(emp?.base_salary || 0) * 0.5 // 50% of salary
if (requestedAmount > maxAdvance) {
  return NextResponse.json(
    { error: `Maximum advance amount is ${maxAdvance} SAR` },
    { status: 400 }
  )
}
```

Effort: 0.5 day

HIGH-7: Commission settlement allowed on unconfirmed/pending sales

Module: Accounting

Files: `src/app/api/accounting/[...segments]/route.ts:575-600`

Impact: Commissions paid out on sales that may later be rejected

How to Fix:

```
// In settleSalesCommissionsFinance(), add a join check:
const { data: commissions } = await adminClient
  .from('agent_commissions')
  .select('*, sim_sales!inner(status)')  // JOIN to check sale status
  .eq('organization_id', auth.orgId)
  .eq('is_paid', false)
  .eq('sim_sales.status', 'confirmed')  // Only confirmed sales
  .in('id', commissionIds)

// Return error for any that aren't confirmed:
const unconfirmed = requestedIds.filter(id => !commissions.find(c => c.id === id))
if (unconfirmed.length > 0) {
  return NextResponse.json({
    error: `${unconfirmed.length} commissions have unconfirmed sales. Confirm sales fir
  }, { status: 400 })
}
```

Effort: 0.5 day

HIGH-8: Chat has NO real-time — messages only appear on manual refresh

Module: Chat

Files: `src/app/(app)/chat/page.tsx`

Impact: Chat is unusable as a communication tool

How to Fix:

```
// Add Supabase realtime subscription in the chat page useEffect:
useEffect(() => {
  if (!activeConversation) return

  const channel = supabase
    .channel(`chat:${activeConversation.id}`)
    .on('postgres_changes', {
      event: 'INSERT',
      schema: 'public',
      table: 'chat_messages',
      filter: `conversation_id=eq.${activeConversation.id}`,
    }, (payload) => {
      // Add new message to state if not from current user
      if (payload.new.sender_id !== currentUser.id) {
        setMessages(prev => [...prev, payload.new])
      }
    })
    .subscribe()

  return () => { supabase.removeChannel(channel) }
}, [activeConversation?.id])

// Also need to enable replication on chat_messages table in Supabase dashboard
```

Effort: 1 day

HIGH-9: Kiosk mode has NO role check — any authenticated user can access

Module: Kiosk

Files: `src/app/(kiosk)/layout.tsx`

Impact: Regular employees can access agent/merchant/promoter kiosk screens

How to Fix:

```tsx
// In src/app/(kiosk)/layout.tsx, add role validation:
import { redirect } from 'next/navigation'

export default async function KioskLayout({ children }) {
  const supabase = createServerClient()
  const { data: { user } } = await supabase.auth.getUser()

  if (!user) redirect('/login')

  // Check if user has a kiosk-eligible role
  const { data: member } = await supabase
    .from('org_members')
    .select('role')
    .eq('user_id', user.id)
    .single()

  const kioskRoles = ['sales', 'onground_agent_manager', 'promoter']
  // Also check if user is a field_agent or merchant
  const { data: agent } = await supabase
    .from('field_agents')
    .select('id')
    .eq('user_id', user.id)
    .maybeSingle()

  const { data: merchant } = await supabase
    .from('merchants')
    .select('id')
    .eq('user_id', user.id)
    .maybeSingle()

  if (!kioskRoles.includes(member?.role) && !agent && !merchant) {
    redirect('/') // Send non-kiosk users back to main app
  }

  return <>{children}</>
}
```

Effort: 0.5 day

HIGH-10: Leave notifications use browser Supabase client — RLS blocks non-admin inserts

Module: Notifications

Files: `src/lib/services/notification-triggers.ts:1,52`

Impact: Leave approval/rejection notifications silently dropped for non-admin actors

How to Fix:

```
1. Move notification triggers to the API layer (server-side):
   - In the leave-requests API route, after approval/rejection succeeds,
     call notification insert using the admin/service_role client

2. In src/app/api/leave-requests/self/route.ts and
   src/app/api/leave-requests/notifications/route.ts:

   // Use admin client for notification inserts:
   const adminClient = createClient(
     process.env.NEXT_PUBLIC_SUPABASE_URL!,
     process.env.SUPABASE_SERVICE_ROLE_KEY!
   )
   await adminClient.from('notifications').insert({
     user_id: targetUserId,
     org_id: orgId,
     title: '...',
     body: '...',
     type: 'leave_decision',
   })

3. Remove the client-side notifyLeaveDecision calls from leave/page.tsx
```

Effort: 0.5 day

HIGH-11: Recruitment pipeline has NO stage ordering — can skip straight to "hired"

Module: Recruitment

Files: `src/app/api/recruitment/applications/[id]/stage/route.ts:44,73-84`

Impact: Recruiter can bypass screening/interview and hire anyone directly

How to Fix:

```
// Define valid transitions:
const STAGE_ORDER = ['submitted', 'screening', 'shortlisted', 'interview', 'technical_t

// In the POST handler, validate transition:
const currentStage = application.status
const requestedStage = body.stage
const currentIndex = STAGE_ORDER.indexOf(currentStage)
const requestedIndex = STAGE_ORDER.indexOf(requestedStage)

if (requestedIndex === -1) {
  return NextResponse.json({ error: 'Invalid stage' }, { status: 400 })
}

// Allow forward by 1 step, or backward to any previous step (for rejection/reconsidera
// But never allow jumping forward more than 1 step
if (requestedIndex > currentIndex + 1) {
  return NextResponse.json({
    error: `Cannot skip stages. Current: ${currentStage}, next allowed: ${STAGE_ORDER[c
  }, { status: 400 })
}

// Special case: 'rejected' can be set from any stage
if (requestedStage === 'rejected') { /* always allowed */ }
```

Effort: 0.5 day

HIGH-12: Five unchecked DB queries in agent dashboard (handleMe)

Module: Sales

Files: `src/app/api/sales/agents/[[...segments]]/route.ts:1405-1432`

Impact: Agent dashboard shows zero data on DB errors instead of error message

How to Fix:

```
// For each query, add error checking:
const simsResult = await adminClient.from('sim_units')...
if (simsResult.error) {
  return NextResponse.json({ error: 'Failed to load SIM data' }, { status: 500 })
}

// Repeat for heldResult, collectionsResult, requestsResult, commissionsResult
// Or use a helper:
function checkResult(result, label) {
  if (result.error) throw new Error(`${label}: ${result.error.message}`)
  return result.data
}

try {
  const sims = checkResult(await adminClient.from('sim_units')..., 'SIMs')
  const held = checkResult(await adminClient.from('...')..., 'Held cash')
  // etc.
} catch (err) {
  return NextResponse.json({ error: err.message }, { status: 500 })
}
```

Effort: 0.5 day

HIGH-13: Auto-post failure logging silently fails — failures are unrecorded

Module: Accounting

Files: `src/lib/accounting/auto-posting.ts:817–821`

Impact: When auto-posting fails, the failure itself fails to log — no audit trail

How to Fix:

```
// In logAutoPostFailure(), don't use source_id (which may be non-UUID):
// Instead, store failure info in a simple log table:

// New migration:
// CREATE TABLE accounting_auto_post_failures (
//   id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
//   organization_id uuid NOT NULL,
//   source_module text NOT NULL,
//   source_identifier text NOT NULL,  -- TEXT, not UUID
//   error_message text,
//   created_at timestamptz DEFAULT now()
// );

// Replace the logAutoPostFailure function:
async function logAutoPostFailure(orgId, sourceModule, sourceId, error) {
  await adminClient.from('accounting_auto_post_failures').insert({
    organization_id: orgId,
    source_module: sourceModule,
    source_identifier: String(sourceId),
    error_message: error?.message || 'Unknown error',
  })
  console.error(`[Auto-Post FAIL] ${sourceModule}/${sourceId}:`, error)
}
```

Effort: 0.5 day

HIGH-14: Chat attachments bucket is PUBLIC — no auth required to download

Module: Chat

Files: `db/schema_v29_chat_module.sql:77-79`

Impact: Anyone with a file URL can download private chat attachments

How to Fix:

```sql
-- Migration to fix:
UPDATE storage.buckets
SET public = false
WHERE id = 'chat-attachments';

-- Add RLS policy for the bucket:
CREATE POLICY chat_attachments_select ON storage.objects
  FOR SELECT USING (
    bucket_id = 'chat-attachments'
    AND auth.role() = 'authenticated'
    AND (storage.foldername(name))[1] = (
      SELECT organization_id::text FROM org_members
      WHERE user_id = auth.uid() LIMIT 1
    )
  );
```

Effort: 0.5 day

HIGH-15: Fire-and-forget audit trail in promoter lead creation

Module: Promoter

Files: `src/app/api/sales/promoter/[[...segments]]/route.ts:521-525`

Impact: Lead creation events and agent notifications silently lost on DB errors

How to Fix:

```ts
// Wrap in try/catch and log failures:
try {
  const { error: eventError } = await insertLeadEvent(...)
  if (eventError) console.error('[Promoter] Failed to log lead event:', eventError)

  const { error: notifyError } = await notifyAgent(...)
  if (notifyError) console.error('[Promoter] Failed to notify agent:', notifyError)
} catch (err) {
  // Don't fail the main request, but log the issue
  console.error('[Promoter] Audit trail error:', err)
}
```

Effort: 0.5 day

HIGH-16: VAT auto-post uses non-existent field, hardcodes 15%

Module: Accounting

Files: `src/lib/accounting/auto-posting.ts:67`

Impact: VAT calculation always uses 15% even for exempt or differently-rated items

How to Fix:

```
// Replace line 67:
// OLD: const taxRate = Number((settings as any).default_tax_rate ?? 15) / 100;

// NEW: Fetch the default tax rate from the tax_rates table:
const { data: defaultRate } = await adminClient
  .from('accounting_tax_rates')
  .select('rate')
  .eq('organization_id', orgId)
  .eq('is_default', true)
  .single()

const taxRate = Number(defaultRate?.rate ?? 15) / 100

// Also fix the double-counting issue:
// If grand_total is VAT-inclusive, extract subtotal correctly:
const subtotal = salesLog.subtotal ?? (salesLog.grand_total / (1 + taxRate))
```

Effort: 0.5 day

3. Medium Severity

Issues that cause incorrect behavior but have workarounds.

MED-1: All "today" calculations use UTC — wrong before 03:00 AST

Files: `src/app/(app)/attendance/page.tsx:71` , `src/lib/services/leaves.ts:11` , multiple others

Fix: Replace `new Date().toISOString().split("T")[0]` everywhere with:

```
// Create a utility: src/lib/utils/date.ts
export function todaySaudi(): string {
  return new Date().toLocaleDateString('en-CA', { timeZone: 'Asia/Riyadh' })
  // Returns 'YYYY-MM-DD' format in Saudi time
}
```

Effort: 0.5 day (find-and-replace across ~15 locations)

MED-2: Leave approval runs via browser Supabase client

Files: `src/lib/services/leaves.ts:176-218`

Fix: Move approval logic to an API route that uses admin client. The superadmin route already does this correctly — replicate the pattern for normal approvals.

Effort: 1 day

MED-3: Overlapping leave requests not blocked — double-deduction possible

Files: `src/app/api/leave-requests/self/route.ts`

Fix: Before inserting, check for overlapping approved/pending requests:

```
const { data: overlap } = await supabase
  .from('leave_requests')
  .select('id')
  .eq('employee_id', employeeId)
  .in('status', ['pending', 'manager_approved', 'approved'])
  .or(`start_date.lte.${end_date},end_date.gte.${start_date}`)
  .limit(1)

if (overlap?.length) {
  return NextResponse.json({ error: 'Overlapping leave request exists' }, { status: 400
}
```

Effort: 0.5 day

MED-4: No check for leave on dates with existing attendance records

Files: `src/app/api/leave-requests/self/route.ts`

Fix: Query `attendance_records` for the date range before accepting the request. Warn but don't block (employee may have checked in briefly before leaving).

Effort: 0.5 day

MED-5: Shift location not cross-checked against check-in location

Files: `src/app/(app)/attendance/page.tsx` , `db/schema_v14_shift_attendance_timing.sql`

Fix: In the server-side check-in (after CRIT-1 fix), verify the employee's active shift location matches their check-in location. Log a warning if mismatched.

Effort: 0.5 day

MED-6: Leave request auto-creates employee records

Files: `src/app/api/leave-requests/self/route.ts:126-152`

Fix: Remove auto-creation. If no employee record exists, return 400 error telling the user to contact

HR.

Effort: 15 minutes

MED-7: days_count at submit vs days deducted at approval use different formulas

Files: `src/app/api/leave-requests/self/route.ts:203` , `src/lib/services/leaves.ts:208-212`

Fix: Use the same `inclusiveDays()` function in both locations. Store the calculated days at submission time and use THAT stored value at deduction time.

Effort: 0.5 day

MED-8: No promoter commission calculated when lead converts

Files: `src/app/api/sales/promoter/[[...segments]]/route.ts`

Fix: When lead status changes to `converted` , calculate and insert a `promoter_commissions` record based on configured rates. Requires a new `promoter_commissions` table or reuse of existing commission tables.

Effort: 2 days

MED-9: Chat 800-message global limit with no pagination

Files: `src/app/api/chat/route.ts:197`

Fix: Change to per-conversation pagination:

```
// Instead of loading ALL messages from ALL conversations:
// Load messages only for the active conversation with cursor-based pagination
.eq('conversation_id', conversationId)
.order('created_at', { ascending: false })
.limit(50)  // Per page
.range(offset, offset + 49)
```

Effort: 1 day

MED-10: Promoter lead conversion doesn't link to SIM stock

Files: `src/app/api/sales/promoter/[[...segments]]/route.ts`
Fix: Add a UI step when converting a lead to link it to a `sim_sales` record via `conversion_sale_log_id`. The agent who sells the SIM should be prompted to select the lead it came from.
Effort: 1 day

MED-11: Final leave notification fires client-side only

Files: `src/app/(app)/leave/page.tsx:1081`
Fix: Move notification creation to the API layer (part of HIGH-10 fix).
Effort: Included in HIGH-10

MED-12: No document expiry notifications exist

Files: None (feature entirely missing)
Fix: Create a cron/scheduled function that:

1. Queries employee documents expiring within 30/14/7 days
2. Creates notification records for the employee and HR
3. Runs daily via Supabase Edge Function or external cron
   Effort: 2 days

MED-13: `completeLesson` accepts any lessonId without course validation

Files: `src/lib/server/lms.ts:799–812`
Fix: Add validation:

```
const { data: lesson } = await adminClient
  .from('lms_lessons')
  .select('id, module:lms_modules!inner(course_id)')
  .eq('id', lessonId)
  .single()

if (lesson?.module?.course_id !== enrollment.course_id) {
  throw new Error('Lesson does not belong to this course')
}
```

Effort: 0.5 day

MED-14: Hire endpoint doesn't verify application is in "offer" stage

Files: `src/app/api/recruitment/hire/route.ts:37-60`

Fix: Add stage check:

```
if (application.status !== 'offer') {
  return NextResponse.json(
    { error: `Cannot hire from stage "${application.status}". Move to "offer" first.` }
    { status: 400 }
  )
}
```

Effort: 15 minutes

MED-15: Hired employee record missing critical fields

Files: `src/app/api/recruitment/hire/route.ts:117-131`

Fix: After hire, redirect HR to a "Complete Employee Profile" wizard that enforces: employee_code, base_salary, manager_id, national_id before the employee can log in.

Effort: 2 days

MED-16: Employee termination doesn't clean up LMS/tasks

Files: Cross-module gap

Fix: Create a `terminateEmployee` function that:

1. Sets employee.status = 'terminated'
2. Cancels active LMS enrollments
3. Unassigns open tasks (or reassigns to manager)
4. Revokes org_member access
5. Fires a notification to HR for final settlement

Effort: 1 day

MED-17: agent_commissions UNIQUE constraint blocks multi-commission model from v53

Files: `db/schema_v47_field_agents.sql` , `db/schema_v53_editable_sim_commission_rules_finance.sql`

Fix:

```
-- Drop the overly-restrictive unique:
ALTER TABLE agent_commissions
  DROP CONSTRAINT IF EXISTS agent_commissions_org_sim_unit_uq;

-- Add a new unique that allows multiple commission types per SIM:
ALTER TABLE agent_commissions
  ADD CONSTRAINT agent_commissions_org_sim_commission_type_uq
  UNIQUE (organization_id, sim_unit_id, commission_type);
```

Effort: 0.5 day

MED-18: Editing partially-paid invoice resets balance_due

Files: `src/app/api/accounting/[...segments]/route.ts:221-235`

Fix:

```
// When recalculating balance_due after edit:
const paidSoFar = Number(invoice.total_amount) - Number(invoice.balance_due)
const newBalance = Math.max(0, newTotal - paidSoFar)
// Use newBalance instead of newTotal for balance_due
```

Effort: 0.5 day

MED-19: Admin role excluded from accounting management inconsistently

Files: `db/schema_v40_accounting_audit_fixes.sql:4-21` , `db/schema_v43_financial_reporting.sql:84`

Fix: Decide one way: either admin can manage accounting or they can't. Update both the DB function and the budget RLS policy to match.

Effort: 15 minutes

MED-20: `advance_requests` amounts use unbounded numeric

Files: `db/schema_v20_advance_requests.sql:5-6`

Fix:

```
ALTER TABLE advance_requests
  ALTER COLUMN requested_amount TYPE numeric(14,2),
  ALTER COLUMN approved_amount TYPE numeric(14,2);
```

Effort: 15 minutes

MED-21: `sim_transfer_type` enum may be missing values

Files: Multiple insert paths in agents/merchants routes

Fix: Verify enum values exist:

```sql
-- Check current values:
SELECT enum_range(NULL::sim_transfer_type);

-- Add missing if needed:
ALTER TYPE sim_transfer_type ADD VALUE IF NOT EXISTS 'assign_to_merchant';
ALTER TYPE sim_transfer_type ADD VALUE IF NOT EXISTS 'return_from_merchant';
```

Effort: 15 minutes

MED-22: No DELETE endpoint for field agents or merchants

Files: Sales API routes

Fix: Add a soft-delete (set status to 'deleted') or a hard-delete with cascade cleanup. Soft-delete is safer:

```
case 'DELETE':
  await adminClient.from('field_agents')
    .update({ status: 'deleted', deleted_at: new Date().toISOString() })
    .eq('id', agentId).eq('organization_id', auth.orgId)
```

Effort: 1 day

MED-23: `merchant_id` required in schema but ignored for merchant callers

Files: `src/lib/merchants/schemas.ts:71`

Fix: Make `merchant_id` optional: `merchant_id: z.string().uuid().optional()`

Effort: 5 minutes

MED-24: `candidate/notifications` has no org_id filter

Files: `src/app/api/candidate/notifications/route.ts:12-18`

Fix: Add `.eq('org_id', orgId)` to the query (or accept cross-org as intentional for candidates and

document it).
Effort: 5 minutes

4. Low Severity

Minor issues — fix when convenient.

#	Module	Issue	Quick Fix
LOW-1	Leave	Sales_Manager approves all Sales employees, not just direct reports	Add `manager_id` filter
LOW-2	Attendance	`within_geofence` / `distance_meters` accepted from client	Fixed by CRIT-1
LOW-3	Attendance	Double-click guard has edge case	Fixed by HIGH-2 (unique constraint)
LOW-4	Leave	Accrual recalc runs every submission	Add a `last_accrual_date` check to skip if same month
LOW-5	Sales	`findAuthUserIdByEmail` scans up to 5000 users	Acceptable for now; add index note
LOW-6	Sales	`sales_shift_prompts.notified_at` never populated	Set it when notification fires client-side
LOW-7	Sales	Fabricated NextRequest in shift-monitor	Refactor to extract params separately
LOW-8	Sales	Reports page sends `unit_price: 0` in reviews	Cosmetic; backend recalculates
LOW-9	Sales	`onground_agent_manager` excluded from shift monitor	Business decision — confirm with client

#	Module	Issue	Quick Fix
LOW-10	Sales	GL auto-post failures silently swallowed	Add `console.error` at minimum
LOW-11	LMS	`requires_quiz_pass` defaults false — easy misconfiguration	Add UI warning when creating course with quizzes
LOW-12	Tasks	Status can revert from completed to pending	Add transition rules if business requires
LOW-13	Tasks	Attachments bucket was public in v19	Fixed in v41 — just verify v41 is deployed
LOW-14	Promoter	Campaign attribution not event-driven	Enhancement for later
LOW-15	Notifications	Leave notification gaps for some roles	Part of HIGH-10 refactor
LOW-16	Chat	Role check for group creation is correct	No action needed
LOW-17	Sales	`hr_admin` removal from field-agent access undocumented	Add release note

5. Implementation Plan

Phase 1: Security & Data Integrity (Week 1)

Goal: Block all paths to data corruption and unauthorized access

Day	Tasks	Effort
1	CRIT-1 + CRIT-2: Server-side attendance API	1.5 days
2	CRIT-3: Atomic leave balance deduction (DB function)	1 day
2	HIGH-1: Fix timezone (15 min)	15 min

Day	Tasks	Effort
2	HIGH-2: Re-add unique constraint	15 min
3	CRIT-4 + CRIT-5: Team creation rollback + email check	1 day
4	CRIT-6 + CRIT-7: Void/cancel for sales	1 day
5	HIGH-9: Kiosk role check	0.5 day
5	HIGH-11: Recruitment stage ordering	0.5 day

Phase 2: Financial Integrity (Week 2)

Goal: Ensure all money is tracked correctly in GL

Day	Tasks	Effort
1	CRIT-8: Fix auto-posting UUID issue	0.5 day
1	CRIT-9: Fix payroll template balance	0.5 day
2	HIGH-5: Commission GL entry	1 day
3	HIGH-6: Advance limit enforcement	0.5 day
3	HIGH-7: Confirm-before-pay for commissions	0.5 day
4	HIGH-16: Fix VAT calculation	0.5 day
4	HIGH-13: Fix failure logging	0.5 day
5	MED-18, MED-19, MED-20: Invoice/permissions/precision fixes	0.5 day

Phase 3: Communication & UX (Week 3)

Goal: Make chat usable, fix notification gaps

Day	Tasks	Effort
1	HIGH-8: Chat realtime subscriptions	1 day
2	HIGH-14: Chat attachments bucket private	0.5 day
2	MED-9: Chat pagination	1 day

Day	Tasks	Effort
3	HIGH-10: Server-side notifications	0.5 day
3	MED-11: Leave notifications to API	0.5 day
4	MED-12: Document expiry notifications	1 day
5	MED-1: Timezone fix across all "today" usages	0.5 day

Phase 4: Business Logic Gaps (Week 4)

Goal: Close remaining logical holes

Day	Tasks	Effort
1	MED-2: Leave approval to API route	1 day
2	MED-3 + MED-4: Overlap check + attendance check	1 day
3	MED-13 + MED-14: LMS validation + hire stage check	0.5 day
3	MED-16: Employee termination cleanup	1 day
4	MED-15: Employee profile completion wizard	1 day
5	MED-17, MED-21, MED-22, MED-23: DB constraint fixes	1 day

Phase 5: Polish (Week 5)

Goal: Handle remaining medium and low items

Day	Tasks	Effort
1-2	HIGH-4: AutoAttendanceRecorder multi-location	0.5 day
1-2	HIGH-3: Device fingerprint improvement	1 day
2-3	HIGH-12 + HIGH-15: Error checking in APIs	1 day
3-4	MED-5, MED-6, MED-7, MED-8: Remaining attendance/leave fixes	1.5 days
5	LOW items: batch fix (timezone, shift monitor, etc.)	1 day

6. Fix Priority Matrix

	IMPACT: Low	Med	High
Easy (< 1h)	LOW-*	MED-6 MED-14 MED-23 MED-24	HIGH-1 HIGH-2
Medium (1-4h)		MED-3 MED-7 MED-17 MED-21	CRIT-4 CRIT-5 HIGH-6 HIGH-7 HIGH-9 HIGH-11
Hard (1+ d)		MED-8 MED-9 MED-12 MED-15 MED-16	CRIT-1 CRIT-3 CRIT-6 CRIT-8 CRIT-9 HIGH-5 HIGH-8

Start from top-right corner (Easy + High Impact) and work down.

7. Auth, RLS & Multi-Tenancy Findings (Addendum)

Completed separately — these are ADDITIONAL findings beyond the numbered items above.

CRIT-10: `is_admin()` function in v41 removes hr_admin — breaks RLS for attendance/leave mutations

Module: Auth / RLS

Files: `db/schema_v41_multi_tenancy_audit_fixes.sql:34-52`

Impact: hr_admin users can no longer update/delete attendance records or approve leave via direct Supabase client

Problem:

The v41 rewrite of `is_admin()` only returns true for `role IN ('admin', 'super_admin')`, removing `hr_admin`. Any RLS policy relying on `is_admin()` for write access (attendance_records UPDATE/DELETE, leave_requests UPDATE/DELETE) silently denies hr_admin users. The API routes work because they use service_role client, but any frontend operation that writes directly via the browser Supabase client breaks for hr_admin.

How to Fix:

```
-- Option A: Add hr_admin back to is_admin():
CREATE OR REPLACE FUNCTION public.is_admin() RETURNS boolean AS $$
BEGIN
  RETURN EXISTS (
    SELECT 1 FROM public.org_members
    WHERE user_id = auth.uid()
      AND role IN ('admin', 'super_admin', 'hr_admin')
  );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER STABLE;

-- Option B (better): Create a separate is_hr() function and update
-- attendance/leave policies to use (is_admin() OR is_hr())
```

Effort: 30 minutes

HIGH-17: Admin can self-escalate role to super_admin via direct DB update

Module: Auth / RLS

Files: `db/schema.sql:523-525`

Impact: Any org admin can become a superadmin by updating their own org_members row

Problem:
The `org_members_update` RLS policy only requires `is_admin()` for USING and WITH CHECK. An admin can issue: `UPDATE org_members SET role = 'super_admin' WHERE user_id = auth.uid()` via the anon Supabase client. This grants them superadmin-level access.

How to Fix:

```
-- Replace the policy with one that prevents role self-escalation:
DROP POLICY org_members_update ON public.org_members;
CREATE POLICY org_members_update ON public.org_members
  FOR UPDATE USING (
    auth.role() = 'service_role'
    OR (public.is_admin() AND user_id != auth.uid())  -- Can't edit own row
  )
  WITH CHECK (
    auth.role() = 'service_role'
    OR (
      public.is_admin()
      AND user_id != auth.uid()
      AND role NOT IN ('super_admin')  -- Can't grant super_admin
    )
  );
```

Effort: 30 minutes

HIGH-18: All accounting routes skip org/user suspension check

Module: Accounting / Auth
Files: `src/lib/accounting/permissions.ts` (all functions)
Impact: Suspended org users retain full accounting access via direct API calls

Problem:
`requireAccountingAccess()` checks authentication and role but never queries `organizations.status` or `profiles.account_status`. A user from a suspended org can still create invoices, journal entries, and process payments.

How to Fix:

```
// In src/lib/accounting/permissions.ts, replace requireAccountingAccess:
export async function requireAccountingAccess(request: NextRequest, action: string) {
  // Use requireTenantContext which already checks suspension:
  const auth = await requireTenantContext(request)

  // Then check accounting-specific roles:
  const allowed = action === 'view' ? viewRoles : manageRoles
  if (!allowed.includes(auth.role)) {
    throw new ForbiddenError('Insufficient accounting permissions')
  }
  return auth
}
```

Effort: 1 day (needs to be applied to all accounting route handlers)

HIGH-19: 9+ recruitment API routes skip suspension + feature flag checks

Module: Recruitment / Auth

Files: All files in `src/app/api/recruitment/*/route.ts`

Impact: Suspended orgs can still post jobs, move candidates, and hire people

Problem:

Each recruitment route defines its own inline `authorizeRecruiter()` that only checks role, not suspension or the `recruitment_enabled` feature flag. Direct API calls bypass the middleware's feature-check.

How to Fix:

```
// Replace all inline authorizeRecruiter() calls with:
import { requireTenantContext } from '@/lib/server/tenant-context'

// In each route handler:
const auth = await requireTenantContext(request, { feature: 'recruitment' })
// This checks: auth, org active, user active, feature enabled, role
```

Effort: 1 day (9 files to update)

MED-25: `hr/employee-accounts` route skips suspension check

Files: `src/app/api/hr/employee-accounts/route.ts:18-41`

Fix: Replace local `requireHrAccess()` with `requireTenantContext(request)`.

Effort: 15 minutes

MED-26: `tasks/messages` and `tasks/updates` routes skip suspension check

Files: `src/app/api/tasks/messages/route.ts`, `src/app/api/tasks/updates/route.ts`

Fix: Replace ad-hoc auth with `requireTenantContext(request)`.

Effort: 15 minutes

MED-27: Superadmin legacy fallback — missing `platform_admins` table grants superadmin access

Files: `src/lib/superadmin/server.ts:34-64`

Fix: Remove the legacy fallback entirely. If `platform_admins` doesn't exist, deny superadmin access:

```
// Remove the catch block that falls back to org_members.role check
// If platform_admins query fails, always deny:
if (paError) {
  return { authorized: false, reason: 'Platform admins table unavailable' }
}
```

Effort: 15 minutes

MED-28: `current_org_id()` RLS helper doesn't check org active status

Files: `db/schema_v41_multi_tenancy_audit_fixes.sql:5-17`

Fix: Either accept this (middleware blocks suspended orgs at HTTP layer) or add a join:

```sql
-- If you want DB-level enforcement:
CREATE OR REPLACE FUNCTION public.current_org_id() RETURNS uuid AS $$
  SELECT om.organization_id FROM public.org_members om
  JOIN public.organizations o ON o.id = om.organization_id
  WHERE om.user_id = auth.uid()
    AND o.status = 'active'
  ORDER BY om.created_at LIMIT 1;
$$ LANGUAGE sql STABLE SECURITY DEFINER;
```

Effort: 30 minutes (but test thoroughly — affects ALL RLS policies)

LOW-18: All roles can access `/tasks` and `/leave` UI paths

Files: `src/lib/rbac.ts:296-298`

Fix: Intentional for self-service. Document as a design decision, not a bug.

Effort: None

LOW-19: Client state lags suspension by up to one token lifetime (~55min)

Files: `src/providers/RoleProvider.tsx:127`

Fix: Accept this. The middleware blocks actual API calls. UX shows logged-in state but all operations fail with 403. Optional improvement: add a periodic org-status check (every 5 min).

Effort: Optional: 1 hour

Updated Implementation Plan — Phase 0 (Add to Week 1)

Priority	Task	Effort
CRIT-10	Fix `is_admin()` to include hr_admin	30 min
HIGH-17	Block role self-escalation in RLS	30 min

Priority	Task	Effort
HIGH-18	Accounting routes use requireTenantContext	1 day
HIGH-19	Recruitment routes use requireTenantContext	1 day
MED-25-28	Replace ad-hoc auth with tenant context	1 hour

These should be done in **Week 1, Day 1** before anything else since they're security-critical.

Notes

- All effort estimates assume one developer familiar with the codebase.
- Critical fixes should be deployed together as a single release to avoid partial-fix states.
- Test plan: Each fix should include a manual test case and, where possible, an integration test against a staging Supabase instance.
- **Tenant isolation is solid** — no path found for org A to access org B data through normal auth. The findings above are about suspension enforcement and role escalation within an org.